

Practical 10

```
In [2]: import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report, roc_auc_score, roc_curve, auc)
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelBinarizer

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset into training and test sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create an SVM classifier
svm_model = SVC(kernel='linear', probability=True)

# Train the model
svm_model.fit(X_train, y_train)

# Predict on the test set
y_pred = svm_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)

# Display accuracy, confusion matrix, and classification report
```

```

print(f"Accuracy: {accuracy}")
print("Confusion Matrix:")
print(conf_matrix)
print("Classification Report:")
print(class_report)

# Perform 5-fold cross-validation
cv_scores = cross_val_score(svm_model, X, y, cv=5)

# Report cross-validation scores
print(f"Cross-Validation Scores: {cv_scores}")
print(f"Mean Cross-Validation Score: {cv_scores.mean()}")

# Compute ROC curve and AUC for multiclass classification (test set only)
lb = LabelBinarizer()
y_bin = lb.fit_transform(y_test) # Use y_test for binary transformation

# Get probabilities of the test set
y_prob = svm_model.predict_proba(X_test)

# Compute the ROC curve and AUC for each class
fpr, tpr, roc_auc = {}, {}, {}

for i in range(len(lb.classes_)):
    fpr[i], tpr[i], _ = roc_curve(y_bin[:, i], y_prob[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC curve
plt.figure()
for i in range(len(lb.classes_)):
    plt.plot(fpr[i], tpr[i], label=f'Class {lb.classes_[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')

```

```
plt.legend(loc="lower right")
plt.show()
```

Accuracy: 1.0

Confusion Matrix:

```
[[10  0  0]
```

```
 [ 0  9  0]
```

```
 [ 0  0 11]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Cross-Validation Scores: [0.96666667 1. 0.96666667 0.96666667 1.]

Mean Cross-Validation Score: 0.9800000000000001

Receiver Operating Characteristic (ROC) Curve

